Constructor g Destructor في C++

محمد الشلبي ازويد ❤ ادعولي وخاصتا ليقولولك مهيمن

Constructor 🔧 Destructor 🗑

تعريف Constructor في C++ واستخداماته

Constructor

دالة خاصة داخل الـ **Class** تنفذ ويتم استدعاؤها تلقائياً عند إنشاء كائن **Object** من الكلاس

استخدامات Constructor

- تخصيص الموارد مثل معالجة الملفات
- تهيئة كياناته اللعبة مثل اللاعبين
- يجب أن يكون اسم **Constructor** نفس اسم الـ **Class**
- عند إنشاء **Object** يتم تشغيل الـ **Constructor** تلقائياً

Class

Constructor

دالة خاصة تنفذ تلقائياً عند إنشاء Object

Code Example

مميزات Constructor وأنواعه

مميزات Constructor

- تعيين **القيم الأولية** لسماته الكائن
- لا يحتوي على **نوع إرجاع** ولا حتى **Void**

يتم تنفيذ Constructor تلقائياً عند إنشاء Object

أنواع Constructor

Default Constructor

Constructor لا يحتوي على parameters

```
class Example {
public:
Example() {}
};
```

Parameterized Constructor

Constructor يقبل قيم (parameters) لإنشاء object بتلك القيم المحددة

```
class Example {
public:
Example(int value) {}
};
```

Constructor Overloading والفرق بين class يحتوي على Constructor و class لا يحتوي

Constructor Overloading

- يسمح **التحميل الزائد** لـ Constructor بأن يكون لها **معلمات متعددة**
- هذا يسمح بإنشاء **object بطرق مختلفة**

Constructor 1

```
()ClassName
{}
```

Constructor 2

```
ClassName(int
{} x)
```

Constructor 3

```
ClassName(int
x, string y)
{}
```

```
} class Student
:private
;string name
;int age
:public
Student() {} // Default
Student(string n) { name = n; } //
Parameterized
Student(string n, int a) { name = n;
age = a; } // Multiple parameters
;{
```

الفرق بين class يحتوي على Constructor و class لا يحتوي

الكلاس الذي يحتوي على Constructor

يتم تهيئة الخصائص في **Constructor** ويتم تنفيذها **بمجرد إنشاء object**

الكلاس الذي لا يحتوي على Constructor

يتم تهيئة الخصائص في دالة **Method** ولا ينفذ إلا **إذا تم استدعاء الدالة**

```
// مع Constructor
} class WithConstructor
:public
()WithConstructor { } // يتم تنفيذها تلقائياً
;value = 10
{
:private
;int value
;{

// بدون Constructor
} class WithoutConstructor
:public
()void initialize { } // يجب استدعاؤها
يدوياً
;value = 10
{
:private
;int value
;{
```

تعريف Destructor ومتى يتم استدعاؤه

Destructor (المدمرات)

دوال خاصة أيضاً يتم استدعاؤها عند **تدمير object** او حذفه

متى يتم استدعاء Destructor

عندما تنتهي الدالة التي تم إنشاء الكائن داخلها

عندما ينتهي البرنامج ويتم تحرير كل الكائنات

Destructor يعمل بشكل تلقائي مثل Constructor ولكن عند نهاية حياة الكائن

مميزات Destructors

- الـ Destructor هو دالة لها اسم Class لكن يسبقها رمز **التلدة (~)**
- لا يوجد في **Destructor overloading**
- ليس لها نوع ارجاع ولا حتى **Void**
- لا يمكن تحميلها بشكل زائد (أي لا يوجد فيها **overloaded destructors**)

```
} class Example
:public
Example() { // Constructor
// يتم تنفيذها عند إنشاء الكائن
{

Example() { // Destructor~
// يتم تنفيذها عند تدمير الكائن
{
;{
```

مميزات Destructor والفرق بين Constructor و Destructor

الفرق بين Constructor و Destructor

الميزة	Constructor	Destructor
التنفيذ	عند **إنشاء** object	عند **تدمير** object
الاسم	نفس اسم الـ Class	اسم الـ Class مع ~ في البداية
التحميل الزائد	**ممكن**	**غير ممكن**
الغرض	تهيئة الكائن	تحرير الموارد

Constructor

يتم تنفيذه عند إنشاء object

Destructor

يتم تنفيذه عند تدمير object

مثال برمجي

```
class Student {
private:
string name;
int age;

public:
// Constructor
Student(string n, int a) {
name = n;
age = a;
cout << "Student created: " << name << endl;
}

// Destructor
~Student() {
cout << "Student destroyed: " << name << endl;
}
};

int main() {
Student s1("Ahmed", 20); // يتم استدعاء Constructor
// ... الكود هنا ...
return 0; // يتم استدعاء Destructor تلقائياً
}
```

نقاط هامة

- Constructor و Destructor يتم استدعاؤهما **تلقائياً**
- Constructor يقبل **معلمات** بينما Destructor لا يقبل
- يمكن أن يكون هناك **عدة Constructors** ولكن **Destructor واحد فقط**